

Medical Imaging I/O and Preprocessing Atoms: A Comprehensive Reference for Pure-Function Implementations

The extraction, transformation, and assembly of medical imaging data represent foundational computational barriers in diagnostic machine learning pipelines. Variations in hardware acquisition, proprietary vendor formats, and the underlying physical properties of imaging modalities necessitate rigorous, mathematically precise preprocessing operations. To construct highly scalable, distributed computational directed acyclic graphs (CDGs) for tasks ranging from neuroimaging feature mapping to macroscopic passenger screening, the computational primitives—referred to hereafter as "atoms"—must be designed as pure functions.

A pure-function architecture demands that operations remain entirely stateless. These atoms must accept already-loaded numerical arrays and explicit metadata in memory, compute the mathematical transformations without relying on external file input/output (I/O), network states, or global variables, and return predictably shaped numerical structures. This decoupling guarantees reproducibility, simplifies unit testing, and allows the seamless distribution of volumetric processing tasks across heterogeneous high-performance computing clusters. Furthermore, it strictly separates the parsing mechanisms—which often require complex library dependencies like `pydicom` or `nibabel`—from the pure mathematical array transformations driven by `numpy` and `scipy`.

The following analysis details best-in-class, pure-function implementations for critical medical image preprocessing stages targeted for the `sciona-atoms-bio` repository. The operations span signal intensity transformations (DICOM windowing, histological stain normalization), dimensional manipulation (3D resampling, 2.5D slice stacking, multi-view projections), and spatial region targeting (brain voxel extraction, connected component noise filtering).

1. DICOM Windowing and Intensity Transformations

Medical imaging formats, specifically Digital Imaging and Communications in Medicine (DICOM), store pixel data representing physical measurements. For example, in Computed Tomography (CT), the data represents tissue density quantified in Hounsfield Units (HU), spanning a wide dynamic range from -1000 (air) to over +3000 (dense bone).¹ To map these wide-dynamic-range physical measurements to an observable grayscale spectrum suitable for convolutional neural networks or human visualization, a Value of Interest (VOI) Look-Up Table (LUT) transformation is applied. This operation is colloquially known as windowing.

Mathematical Framework for Windowing

The DICOM standard defines specific continuous algebraic functions to execute this mapping,

relying on two primary metadata attributes: Window Center (c) and Window Width (w).⁴ The transformation interpolates the input pixel value (x) into an output range, typically bounded by $y_{min} = 0$ and $y_{max} = 255$ for 8-bit unsigned integer representations. The standard defines two distinct linear transformation functions based on the VOI LUT Function attribute: LINEAR and LINEAR_EXACT.⁴

The historical default LINEAR function includes a fractional shift to center the continuous interpolation perfectly across integer boundaries.⁴ The mathematical definition applies a conditional piecewise clamping:

- If $x \leq c - 0.5 - \frac{w-1}{2}$, the output is y_{min} .
- If $x > c - 0.5 + \frac{w-1}{2}$, the output is y_{max} .
- Otherwise, the output is computed as:

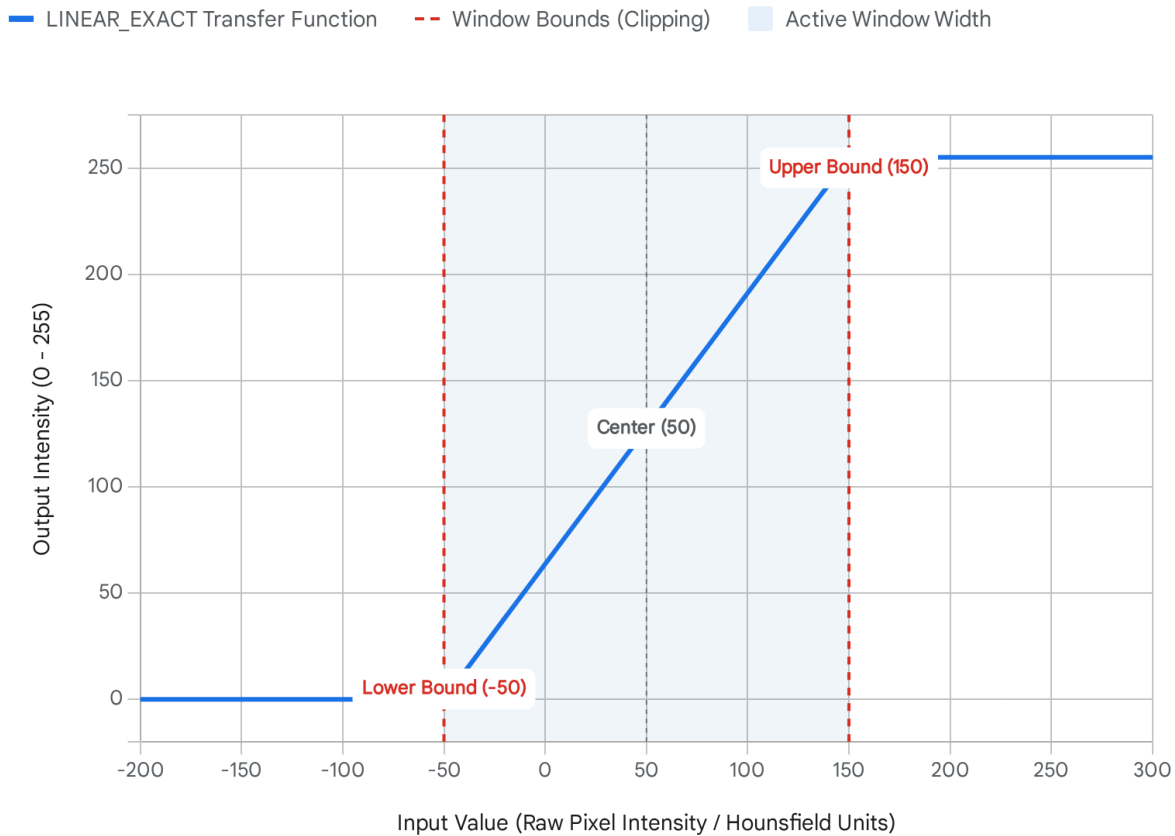
$$y = \left(\frac{x - (c - 0.5)}{w - 1} + 0.5 \right) \times (y_{max} - y_{min}) + y_{min}$$

Introduced to resolve ambiguity in calculating exact bounds without floating-point shifts, the LINEAR_EXACT function simplifies the bounds and the slope.⁴ Its piecewise definition is:

- If $x \leq c - \frac{w}{2}$, the output is y_{min} .
- If $x > c + \frac{w}{2}$, the output is y_{max} .
- Otherwise, the output is computed as: $y = \left(\frac{x - c}{w} + 0.5 \right) \times (y_{max} - y_{min}) + y_{min}$.

Implementing these transformations strictly via numpy ensures the operation remains a fast, vectorized, pure function. By passing a pre-loaded numeric array along with explicit center and width variables, the atom avoids the overhead of I/O parsing libraries.³ Vectorization is achieved via numpy.clip to enforce the boundary conditions, followed by standard array arithmetic.

DICOM Value of Interest (VOI) LUT Transfer Function



The transformation maps raw input values onto an 8-bit output scale (0-255). Values below the calculated lower bound are clamped to black, while values above the upper bound are clamped to white. The slope of the active window is determined by the Window Width.

Data sources: [Innolitics \(DICOM Standard Browser\)](#), [NEMA DICOM Standard](#)

Edge Cases, Contracts, and Data Structures

Several critical edge cases govern the stability of this atom. The DICOM specification explicitly forbids a Window Width of $w = 0$.⁴ A zero width would mathematically trigger a division by zero error in the slope calculation. If $w = 1$ under the LINEAR function, division by zero is mathematically avoided because the continuous segment interval becomes infinitely small; it acts as a binary threshold.⁴ However, computational implementations must handle $w = 1$ defensively, often routing it to a discrete thresholding operation to prevent floating-point instability.

Another vital edge case involves the photometric interpretation of the dataset. DICOM windowing is fundamentally designed for MONOCHROME1 and MONOCHROME2 images.⁴ Providing a multi-channel RGB array (e.g., from an ultrasound sequence) violates the contract of the VOI LUT, as color channels should not be independently mathematically clamped using Hounsfield algorithms.

Furthermore, the input array may contain NaN or infinite values if subjected to prior aggressive spatial masking. A robust pure function must safely filter or cast these anomalies before applying the piecewise transformation to prevent propagation of NaN outputs into downstream convolutional networks.

Edge Case Scenario	Mathematical Consequence	Required Atom Handling Strategy
Window Width = 0	Division by zero in slope calculation.	ValueError exception; violates DICOM standard.
Window Width = 1	Infinite slope; mathematically a step function.	Direct binary thresholding at Window Center.
Multi-channel (RGB) Input	Incorrect independent color shifting.	Contract rejection; enforce ndim == 2 or 3 (if volumetric grayscale).
NaN / Inf in Input Array	Propagation of non-finite floats to output.	Pre-computation masking using numpy.nan_to_num.

Atom Implementation Details

Name: dicom_window
Description: Apply DICOM window center and width to an already-loaded pixel array and return clipped/scaled intensities. Supports both LINEAR and

LINEAR_EXACT formulations.

Source:

<https://dicom.innolitics.com/ciods/parametric-map/parametric-map-multi-frame-functional-groups/52009229/00289132/00281056>

License: Public Domain (DICOM Standard specification)

Concept type: signal_transform

Signature: (pixel_array: NDArray, window_center: float, window_width: float,
exact: bool = False, y_min: float = 0.0, y_max: float = 255.0) -> NDArray

Pure function boundary: strictly numeric inputs (unprocessed HU or scaled pixels) and float metadata parameters in; mapped numeric array out. No pydicom dataset objects, no disk access.

Contracts:

- require: window_width > 0
- require: pixel_array is numeric (int or float)
- ensure: result.shape == pixel_array.shape
- ensure: np.min(result) >= y_min and np.max(result) <= y_max

Witness: Create a 1D array np.array([-100, 0, 50, 100, 200]). With center=50 and width=100 using exact=True. Ensure -100 maps to 0, 0 maps to 0, 50 maps to 127.5, 100 maps to 255, and 200 maps to 255.

Dependencies: numpy

CDG stages covered: rsna_pe/dicom_windowing, rsna_mammography/windowing, rsna_cervical/dicom_windowing

2. 3D Volume Resampling and Coordinate Mapping

Volumetric medical scans, such as CT and MRI sequences, capture biological structures across discrete spatial grids. However, the physical distance between adjacent voxels—known as the pixel spacing—frequently varies between imaging centers and is heavily anisotropic.⁸ For

instance, a cervical spine CT might possess a highly resolved axial plane (e.g., 0.5×0.5

mm) but a much coarser longitudinal z-axis slice thickness (e.g., 3.0 mm).⁹ To standardize spatial dimensions for 3D convolutional neural networks (such as the 3D U-Nets used in RSNA Cervical Spine or MICCAI Brain Tumor segmentation), raw volumes must be mathematically resampled to a consistent, isotropic target resolution.

Algorithmic Approaches: Spline Filters vs. Coordinate Mapping

Within the Python ecosystem, the scipy.ndimage module provides two distinct primary functions for dimensional resizing: zoom and map_coordinates.⁸

The scipy.ndimage.zoom function operates by applying a designated scaling multiplier directly to each axis of the multidimensional array, utilizing B-spline interpolation of a requested order.¹²

While syntactically simple and efficient for minor upscaling, zoom suffers from a specific architectural limitation in medical imaging: it relies on nearest-neighbor-like downsampling

algorithms when the zoom factor dictates severe compression.¹² Rather than performing area-averaging or block-averaging across the decimated voxels, zoom samples discrete points, a behavior that significantly exacerbates high-frequency noise and introduces severe spatial aliasing when moving from high-resolution millimeter grids to lower-resolution neural network inputs.

Conversely, `scipy.ndimage.map_coordinates` offers a highly generalized, robust alternative.⁸ It accepts an input array and an explicit array of geometric coordinates representing the new spatial grid. It then evaluates the input image at those precise sub-pixel locations using continuous interpolation.⁸ This approach fundamentally decouples the scaling math from the interpolation math, allowing the application of complex, non-linear coordinate mappings or precise affine transformations that define the relationship between the patient's physical millimeter space and the raw voxel index space.⁸

To compute the necessary coordinate grid for `map_coordinates` to perform simple anisotropic resampling, a regular mathematical meshgrid of indices corresponding to the target shape must be generated.¹⁷ These output indices are subsequently scaled by the ratio of the target spacing to the current spacing, projecting them backwards into the coordinate space of the original volume.¹⁸

Mathematical Definition of the Voxel Mapping

Let V_{in} represent a 3D medical volume with a shape structure of (Z_{in}, Y_{in}, X_{in}) and corresponding physical spacing $S_{in} = (s_z, s_y, s_x)$. Let the desired target physical spacing be $S_{out} = (t_z, t_y, t_x)$.

First, the target array shape is computed to preserve the overall physical field of view. To avoid fractional array dimensions, the calculation is floored:

$$D_{out} = \lfloor D_{in} \times \frac{S_{in}}{S_{out}} \rfloor$$

A dense 3D coordinate grid G is generated encompassing the bounds of D_{out} . This grid represents every voxel index in the new target volume. The grid is then transformed by the scaling matrix $\frac{S_{out}}{S_{in}}$ to locate the corresponding continuous sampling points mapped within V_{in} . Finally, `map_coordinates` interpolates the intensities at these floating-point coordinates.⁸

Interpolation order is an essential contract variable. For categorical segmentations (such as binary organ masks or multi-class tumor regions), the interpolation order must strictly be 0 (nearest neighbor). Utilizing higher-order splines on a binary mask generates continuous fractional labels along the boundaries, corrupting the discrete categorical nature of the data. Conversely, for continuous intensity volumes (e.g., T1-weighted MRI or HU-scaled CT), higher-order B-spline interpolation (typically order 1 for linear or 3 for cubic) is mandatory to

preserve biological anatomical gradients and edge continuity without introducing voxelated artifacting.⁹

Data Type	Required Interpolation Order	Interpolation Method	Justification
Binary/Categorical Masks	0	Nearest Neighbor	Prevents fractional class generation at boundaries.
Continuous Intensity Volumes	1	Trilinear	Fast, hardware-efficient continuous spatial blending.
High-Fidelity Intensity Volumes	3	Cubic B-Spline	Maximum preservation of high-frequency anatomical gradients.

Atom Implementation Details

Name: `resample_volume`

Description: Resample a 3D medical volume to a target physical spacing using continuous coordinate mapping, handling both isotropic and anisotropic scaling.

Source: https://bic-berkeley.github.io/psych-214-fall-2016/map_coordinates.html

License: BSD (SciPy)

Concept type: `signal_transform`

Signature: `(volume: NDArray, current_spacing: tuple[float, float, float], target_spacing: tuple[float, float, float], order: int = 1) -> NDArray`

Pure function boundary: Accepts raw volume array and strictly numerical spacing tuples. Returns new grid array. Relies purely on numpy meshgrid and scipy interpolation.

Contracts:

- require: `volume.ndim == 3`

- require: all($s > 0$ for s in `current_spacing`)
- require: all($t > 0$ for t in `target_spacing`)
- require: order in
- ensure: `result.shape` corresponds to `volume.shape * current_spacing / target_spacing`

Witness: Pass a 2x2x2 volume of ones, spacing (1,1,1) target spacing (0.5,0.5,0.5).
 Assert output is 4x4x4 array of ones.

Dependencies: numpy, scipy.ndimage (map_coordinates)

CDG stages covered: rsna_cervical/volume_resampling, passenger_screening/resampling

3. Histological Stain Normalization: The Macenko Method

Within the subfield of computational pathology, Whole Slide Images (WSIs) stained with Hematoxylin and Eosin (H&E) exhibit extreme colorimetric variability.²⁰ Differences in chemical stain manufacturers, laboratory incubation protocols, section thickness, and slide scanner hardware result in severe batch effects.²⁰ Hematoxylin typically binds to nucleic acids (staining nuclei dark blue/purple), while Eosin binds to proteins (staining cytoplasm and extracellular matrix varying shades of pink). When machine learning models are trained on data from a single laboratory, this color instability severely degrades the generalization capabilities of deep learning models applied to broader tasks like HubMap Human Protein Atlas (HPA) segmentation.

To standardize the colorimetric distribution across cohorts, researchers employ normalization techniques. While deep generative models such as CycleGANs are occasionally utilized for complex stain translation (mapping from H&E to completely different stains like Masson's Trichrome), they are heavily state-dependent.²³ CycleGANs require loading massive sets of trained PyTorch or TensorFlow neural network weights, managing GPU VRAM state, and executing stochastic inference passes. Consequently, they inherently violate the design philosophy of a stateless, pure-function preprocessing atom repository.

Instead, the Macenko stain normalization algorithm provides a deterministic, matrix-factorization-based mathematical correction that perfectly aligns with pure-function constraints.²⁴

The Optical Density Transform and SVD Plane

The Macenko method operates on the fundamental biophysical principle that the absorption of light by chemical stains follows the Beer-Lambert law. In the standard RGB digital color space captured by scanners, the physical mixtures of these stains appear mathematically non-linear.²⁵

Therefore, the first operational step is mapping the RGB image array I (normalized to float values between 0 and 1) into an Optical Density (OD) space using a logarithmic transform:

$$OD = -\log_{10}(I + \epsilon)$$

Where ϵ is a small constant (e.g., 1e-6) integrated defensively to prevent undefined $\log(0)$ operations in pure black pixels. In this transformed OD space, the relationship between the physical stain concentrations and the observed color vectors becomes mathematically linear.²⁵

Once the data is mapped into OD space, background pixels (which represent the glass slide, appearing bright white in RGB and demonstrating near-zero OD) must be filtered out. A scalar threshold (typically $OD < 0.15$) is applied to isolate actual biological tissue.²⁰ Including the background in the calculation would allow uninformative variance to heavily skew the subsequent vector estimation.

The core ingenuity of the Macenko algorithm is identifying the primary stain vectors mathematically. By applying Singular Value Decomposition (SVD) or Principal Component Analysis (PCA) to the filtered matrix of OD pixels, the algorithm identifies a 2D plane that encapsulates the maximum variance of the stain concentrations.²⁴ The OD pixels are then projected onto these first two singular vectors, creating an SVD plane.

Robust Angular Thresholding and Matrix Estimation

Once the pixel data is projected onto the 2D SVD plane, every single tissue pixel is geometrically characterized by its angle ϕ relative to the singular vectors. Due to their distinct chemical binding properties, Hematoxylin and Eosin inherently exist at the extreme opposing angles of this distribution.²⁰

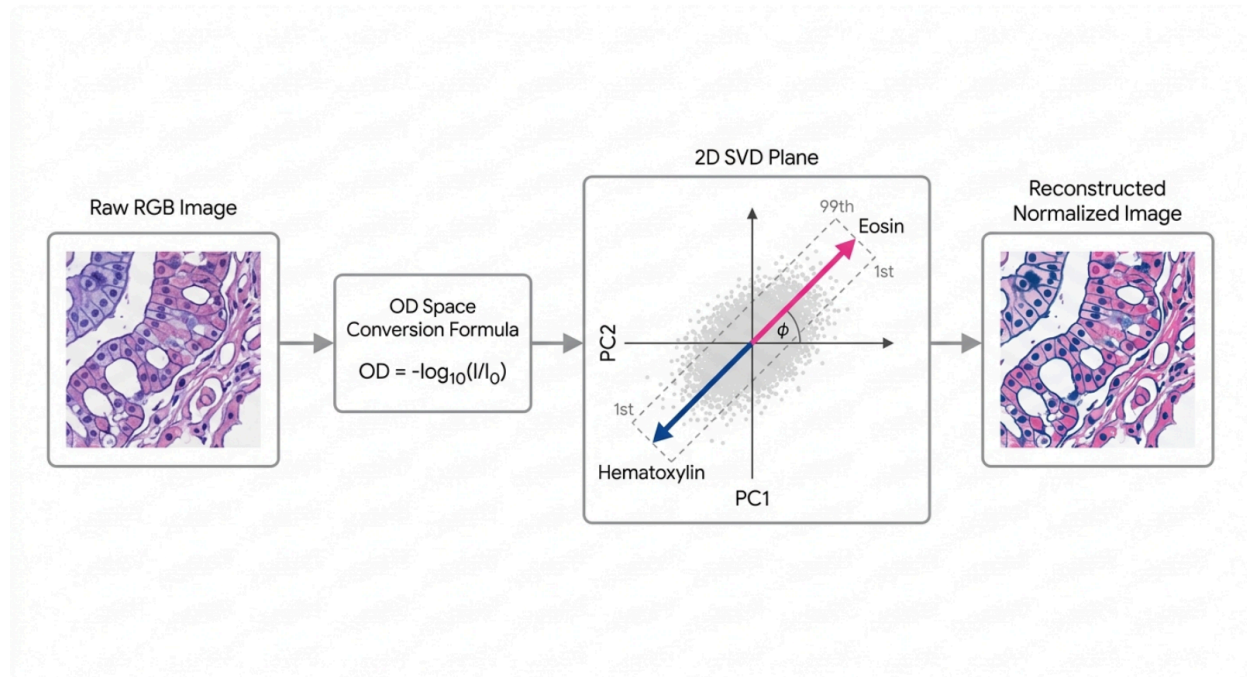
However, finding the absolute minimum and maximum angles geometrically is highly unstable. Sensor noise, debris, or over-stained artifacts on the glass slide can create spurious extreme vectors. Macenko et al. elegantly solve this by calculating the robust extremes using the α -th and $(100 - \alpha)$ -th percentiles of the angle ϕ (empirically tuned, where $\alpha = 1$).²⁰

These two extreme angles—representing the 1st and 99th percentiles—are mapped back from the 2D SVD plane into the full 3D OD space.²⁵ This generates the characteristic 3×2 Optical Density Matrix (ODM), which represents the mathematically pure stain vectors for that specific image.²⁵

By computing the inverse (or pseudo-inverse) of this ODM , the algorithm separates the source image into two independent concentration maps: one purely representing Hematoxylin, and one representing Eosin. Normalization is finally achieved by scaling these individual concentration maps to match the concentration profile of a defined reference standard image, and recombining them through an exponential mapping back to standard RGB space for

downstream viewing or CNN ingestion.²⁴

Algorithmic Pipeline of Macenko Stain Normalization



The pipeline transforms RGB pixels into logarithmic Optical Density space to linearize stain absorption. Singular Value Decomposition (SVD) identifies the primary variance plane. Robust angular percentiles (1st and 99th) isolate the true Hematoxylin and Eosin vectors, ignoring artifact noise. Concentrations are scaled and reconstructed.

Atom Implementation Details

Name: `macenko_stain_vectors`

Description: Computes the 3x2 stain matrix (Optical Density Matrix) for a given RGB image using the Macenko SVD and robust angular percentile method.

Source: <https://www.cs.unc.edu/~mn/sites/default/files/macenko2009.pdf>

License: MIT (Implementation standard via StainTools)

Concept type: `signal_transform`

Signature: (image: `NDArray`, alpha: float = 1.0, beta: float = 0.15) -> tuple

Pure function boundary: Accepts raw RGB uint8 or float arrays; returns the stain matrix and concentration profiles without accessing disk reference

images.

Contracts:

- require: `image.ndim == 3` and `image.shape[-1] == 3`
- require: `0 <= alpha <= 50`
- require: `beta > 0`
- ensure: `stain_matrix.shape == (3, 2)`

Witness: Provide an image containing varying shades of pink and blue. Output stain vectors should heavily weight the Red and Blue channels independently, and the concentration profile must strictly be non-negative.

Dependencies: `numpy`

CDG stages covered: `hubmap_hpa/stain_normalization`

4. Multi-View 2D Projection from Volumetric Data

For rapid macroscopic screening algorithms—such as those required in the Passenger Screening Algorithm Challenge—processing entire dense 3D spatial volumes using full 3D convolutions is often computationally prohibitive due to hardware constraints and real-time latency requirements.²⁶ An effective alternative strategy is projecting the dense 3D volume into multiple 2D views from varying perspectives. This dimensional reduction allows fast, highly optimized 2D architectures (like ResNet or DenseNet) to extract features efficiently.

The Maximum Intensity Projection (MIP) is a technique highly effective for identifying dense anomalies, foreign objects, or highly scattering surfaces within a volume.²⁷ A multi-view MIP requires mathematically rotating the 3D volume array along a defined central axis by a given angle, then collapsing the volume along the designated viewing axis by extracting the maximum voxel intensity across the depth dimension.²⁸

Computational Overhead and Rotation Mechanics

While conceptually simple, 3D image rotation is a computationally expensive operation. Implementations utilizing `scipy.ndimage.rotate` apply a continuous rotation matrix to the underlying coordinate space and perform spline interpolation across the entire multi-million element array.⁹ Because the subsequent projection step instantly discards the depth dimension via a simple `numpy.max(axis=0)` reduction, rotating the entire discrete volume at high fidelity introduces massive, unnecessary computational overhead.

However, keeping the atom as a pure function relying exclusively on standard scientific libraries mandates the use of `scipy.ndimage.rotate` followed by the projection. Significant algorithmic optimization can be achieved by deliberately lowering the interpolation order to 0 (nearest neighbor) or 1 (linear) during the rotation phase.⁹ The maximum selection operation inherently destroys sub-pixel spatial precision by collapsing the axis, rendering the computation of higher-order cubic splines wasteful and inefficient.

Atom Implementation Details

Name: `max_intensity_projection`

Description: Generates a 2D maximum intensity projection from a 3D volume at a specified rotational angle along a target axis.

Source:

<https://stackoverflow.com/questions/48178916/maximum-intensity-projection-from-image-stack>

License: BSD (SciPy)

Concept type: `data_assembly`

Signature: `(volume: NDArray, angle: float, rotation_axes: tuple[int, int] = (1, 2), projection_axis: int = 0) -> NDArray`

Pure function boundary: Pure array manipulation mapping 3D to 2D. No plotting, no GPU state required.

Contracts:

- require: `volume.ndim == 3`
- require: `len(rotation_axes) == 2`
- ensure: `result.ndim == 2`

Witness: Given a 3D zero array with a single internal 1. Rotate by 0, 90, 180 degrees; verify the maximum projection accurately shifts the location of the 1 in the resulting 2D matrix.

Dependencies: `numpy`, `scipy.ndimage (rotate)`

CDG stages covered: `passenger_screening/multi_view_projection`

5. 2.5D Slice Extraction and Spatial Context Assembly

Despite the physiological reality that biological tissues are inherently volumetric, current graphics processing unit (GPU) memory capacities place hard limits on the batch size and spatial resolution achievable by native 3D CNN architectures during training.¹⁰ Attempting to process entire high-resolution MRIs simultaneously forces severe downsampling, which destroys vital diagnostic micro-textures. To circumvent this, the "2.5D" approach was pioneered within medical imaging.³⁰

Instead of processing massively downsampled full 3D blocks, or processing completely isolated 2D slices that lack spatial context, the 2.5D methodology extracts a target central 2D slice and stacks a defined number of physically adjacent slices along the channel dimension.³⁰ For example, a common configuration inputs a stack of 5 or 7 consecutive slices into a network to denoise or segment only the middle slice.³⁰ This structural manipulation grants the 2D CNN (such as an adapted U-Net or SegFormer) critical out-of-plane spatial context—allowing it to perceive Z-axis gradients, tissue continuity, and anatomical boundaries—while retaining the highly favorable computational efficiency and low memory footprint of a 2D network.³⁰

Boundary Handling and Array Slicing Mechanics

The extraction algorithm is fundamentally a memory indexing operation: given a target spatial slice index Z , the atom must return a tensor composed of slices $[Z - N, \dots, Z, \dots, Z + N]$. The dimension of the output tensor is $(2N + 1, Y, X)$.

The primary mathematical complexity arises at the boundaries. When extracting a 2.5D stack near the physical outer edges of the scan (e.g., if $Z < N$ or $Z + N \geq Z_{max}$), the simple indexing operation will attempt to access out-of-bounds array memory. To satisfy the strict static shape contracts required by downstream neural network tensor inputs, the atom must dynamically pad the volume.³³

The preferred boundary handling mechanism in medical imaging is reflection or symmetric padding (e.g., mode='reflect' or mode='nearest' in numpy.pad) rather than simple zero-padding. Zero-padding introduces sharp, artificial gradients (a sudden drop from tissue intensity to absolute zero) at the volume edges.³⁶ These artificial cliffs can trigger false-positive edge detection filters within the first convolutional layer of the network, degrading performance.

An extreme edge case involves single-slice volumes (e.g., 2D X-rays mistakenly formatted as thin 3D volumes). If the total volume depth is 1, but $N = 2$ adjacent slices are requested, the padding logic must be robust enough to reflect the single slice repeatedly across the channel axis to fulfill the shape contract, essentially falling back to a pure 2D processing mode without crashing the CDG.

Padding Strategy	Mathematical Behavior	Impact on Convolutional Networks
Zero Padding	Fills out-of-bounds indices with 0.	High risk of artificial edge artifacts; disrupts normalization layers.
Nearest Padding	Duplicates the exact edge slice infinitely.	Low risk; preserves local intensity, but eliminates gradients.
Reflect Padding	Mirrors the internal slices	Optimal ; preserves natural

	outward.	texture gradients and prevents sharp cutoffs.
--	----------	---

Atom Implementation Details

Name: `extract_25d_slices`

Description: Extracts a 2.5D representation by stacking a central slice with its adjacent neighboring slices along the channel axis. Automatically handles volume boundaries via reflective padding.

Source: <https://www.mdpi.com/2306-5354/9/9/467>

License: Public Domain (Algorithmic Concept)

Concept type: `data_assembly`

Signature: `(volume: NDArray, center_idx: int, num_adjacent: int, axis: int = 0) -> NDArray`

Pure function boundary: Executes strictly via numpy slicing and padding. No reliance on PyTorch/TensorFlow tensor classes.

Contracts:

- require: `volume.ndim >= 3`
- require: `num_adjacent >= 0`
- require: `0 <= center_idx < volume.shape[axis]`
- ensure: `result.shape == (2 * num_adjacent) + 1`

Witness: Pass an array shaped (10, 100, 100) with `center_idx=0` and `num_adjacent=2`. Assert the output shape is (5, 100, 100) and that the first two slices reflect the geometry of the available volume.

Dependencies: `numpy`

CDG stages covered: `rsna_cervical/slice_extraction`, `dfl/slice_extraction`

6. Volumetric Bounding Box Extraction and Brain Voxel Cropping

When analyzing complex neurological imaging modalities (such as fMRI or structural MRI formats heavily utilized in the TReNDS Neuroimaging dataset ³⁷), the target organ typically occupies only a fraction of the total spatial scan volume. The surrounding field of view contains background air, skull, meninges, and neck tissues. These extraneous regions contribute zero diagnostic information, heavily dilute computational resources, and can introduce bias in global

normalization functions.⁴⁰

To optimize subsequent processing steps, the volume is cropped to a tightly fitted 3D bounding box surrounding a predetermined binary brain mask. While applying a direct element-wise boolean multiplication mask successfully zeroes out the background, it retains the original massive tensor shape, ultimately failing to improve GPU memory consumption or processing speed.⁴² A true spatial crop physically reduces the array dimensions to the minimum viable footprint.⁴²

Index Extraction Logic and Margin Application

The pure function implementation leverages `numpy.argwhere` to map the non-zero elements of the binary mask to their exact Cartesian coordinates. For a 3D mask, this yields an array of shape $(K, 3)$, where K is the number of active, non-zero voxels.⁴³

The spatial extremes of the brain structure are computed by taking the column-wise minimum and maximum of these discrete coordinates:

$$\mathit{min}_{coord} = \min(\mathit{coords}, \mathit{axis} = 0)$$

$$\mathit{max}_{coord} = \max(\mathit{coords}, \mathit{axis} = 0)$$

To avoid cropping too tightly and accidentally discarding boundary meninges or cortical surfaces that may contain slight misalignments, a uniform voxel margin is added to the bounding box. This margin expansion must be mathematically clamped using `numpy.clip` or conditional max/min functions to guarantee the expanded coordinates do not exceed the absolute boundaries of the original array.⁴³

An edge case to handle is the completely empty mask. If the mask contains zero active voxels, `argwhere` returns an empty set. The contract must either reject the input or return the original full volume, as a zero-volume crop is mathematically invalid for downstream image processing. Once the valid bounds are defined, the algorithm creates standard Python slice objects to dynamically sub-select the memory of the volume.

Atom Implementation Details

Name: `crop_to_mask`

Description: Crops a 3D volumetric array to the minimum bounding box encompassing all non-zero elements of a provided binary mask, with an optional margin.

Source:

<https://stackoverflow.com/questions/63044577/crop-3d-image-based-on-2d-mask-in-python-using-numpy-and-opencv>

License: MIT (Implementation logic standard)

Concept type: data_assembly

Signature: (volume: NDArray, mask: NDArray, margin: int = 0) -> tuple]

Pure function boundary: Purely index-driven array sub-selection. Returns both the cropped volume and the exact slice coordinates to permit reversible mapping if downstream reconstruction is required.

Contracts:

- require: volume.shape == mask.shape
- require: margin >= 0
- require: np.any(mask) is True (mask cannot be completely empty)
- ensure: result_volume.size <= volume.size

Witness: Pass a 10x10x10 array with a mask that is 1 only at (5,5,5). Margin 0 should return a 1x1x1 array. Margin 1 should return a 3x3x3 array.

Dependencies: numpy

CDG stages covered: trends_neuroimaging/voxel_extraction

7. Morphological Filtering and Connected Component Analysis

Modern segmentation architectures, despite deep contextual training, regularly output probability masks peppered with high-frequency noise. This noise manifests as isolated pixels or microscopic anatomical "islands" falsely classified as the target pathology. In tasks like passenger body zone segmentation, where identifying contiguous threat objects is paramount, these fragmented false positives degrade system reliability.⁴⁴ To stabilize clinical or security quantification, these raw masks require post-processing through morphological filtering.

The objective is to eliminate small, disconnected components while preserving the morphology of the primary continuous target structures.

Component Labeling and Frequency Counting

The optimal pure-function approach avoids slow recursive flooding algorithms and leverages the highly optimized `scipy.ndimage.label` function.⁴⁶ This algorithm scans the multidimensional binary array and assigns a unique integer identifier to every isolated cluster of non-zero pixels based on defined structural connectivity metrics.⁴⁶ In 2D, this is typically 4-connectivity or 8-connectivity. In 3D biological volumes, 6-connectivity (face-sharing), 18-connectivity (edge-sharing), or 26-connectivity (vertex-sharing) define how voxels group into a continuous object.⁴⁶

Once uniquely labeled, the physical volume of each cluster must be determined. While `scipy.ndimage.sum` can calculate these areas across the labels⁴⁷, utilizing `numpy.bincount`

executed directly on the flattened label array operates with extreme performance. It yields an array where the index corresponds to the cluster label, and the value represents the absolute voxel count.

The filtering mechanism then thresholds the output of `bincount` against the required `min_size` parameter. A boolean lookup table is created, isolating only the indices of components that satisfy the size requirement. Mapping this continuous lookup table back through the labeled array instantly zeroes out all microscopic false-positive clusters.⁴⁸

Atom Implementation Details

Name: `filter_small_components`

Description: Removes isolated connected components from a binary mask that contain fewer voxels than a defined threshold.

Source:

<https://stackoverflow.com/questions/53576830/how-to-find-the-largest-connected-region-using-sciPy-ndimage>

License: BSD (SciPy)

Concept type: `signal_transform`

Signature: `(mask: NDArray, min_size: int, structure: NDArray = None) -> NDArray`

Pure function boundary: Deterministic array processing. No threshold learning or global dependencies.

Contracts:

- require: mask is a boolean or binary integer array
- require: `min_size > 0`
- ensure: `result.shape == mask.shape`
- ensure: result is strictly binary (0 or 1)

Witness: Provide a 10x10 zero array with a 3x3 block of ones and a completely isolated single one. Set `min_size=5`. Ensure the resulting array maintains the 3x3 block but obliterates the isolated one.

Dependencies: `numpy`, `scipy.ndimage` (`label`)

CDG stages covered: `passenger_screening/zone_segmentation`

8. Proprietary Format Conversion: mmWave to Numpy (.a3d)

The Passenger Screening Algorithm Challenge relies on High-Definition Advanced Imaging Technology (HD-AIT) millimeter wave scanners. Unlike standard medical formats, this target modality outputs raw scatter data in proprietary binary formats, most notably the .a3d Combined

Image 3D file.⁴⁹ Each .a3d file is approximately 330MB, representing a massive array of signal returns.⁴⁹

Byte Streaming and Shape Reconstitution

Unlike DICOM, which contains robust, highly standardized header metadata detailing byte allocation, photometric interpretation, and transfer syntax, the .a3d format operates as a raw, headless binary stream of 16-bit unsigned integers.⁴⁹ The structural integrity of the volume depends entirely on implicit, hardcoded data ordering specifications: the data is laid out as Y, Z, X mapping to Height, Depth Axis, and Horizontal Axis, with fixed sampling dimensions $N_y = 660, N_z = 512, N_x = 512$.⁴⁹

Axis Descriptor	Stride / Offset	Number of Samples (N)	Physical Length
X-Axis (Horizontal)	1	512	1.0 meters
Z-Axis (Depth)	512	512	1.0 meters
Y-Axis (Height)	262,144	660	2.0955 meters

To parse this without relying on external file I/O operations inside the pure function boundary, the atom must accept the raw loaded byte stream (as a Python bytes object) rather than a file path string. This enforces the stateless contract. `numpy.frombuffer` rapidly converts this sequence into a 1D contiguous array of `uint16` structures, parsing the memory directly without heavy allocation overhead.⁴⁹

Following memory parsing, the flat array is strictly reshaped to (660, 512, 512) as defined by the hardware stride constraints.⁴⁹ Immediate transposition via `numpy.transpose` reorders the axes from the scanner-specific (Y, Z, X) to a standard medical spatial formulation (Z, Y, X) . This reordering is crucial to ensure downstream mathematical compatibility with the previously defined 2.5D slice extractors and resampling algorithms, which expect depth to precede height and width. Furthermore, the format documentation mandates that exact pixel intensities are achieved by multiplying the integer representation by a `data_scale_factor` parsed separately from the header, upgrading the data type to floating point.⁴⁹

Atom Implementation Details

Name: `parse_a3d_volume`

Description: Converts a raw binary byte stream of mmWave.a3d data into a spatially structured, transposed, and scaled 3D numpy array.

Source: <https://www.kaggle.com/c/passenger-screening-algorithm-challenge/data>

License: Public Domain (Kaggle dataset specification)

Concept type: `data_extraction`

Signature: (`byte_stream: bytes, data_scale_factor: float, shape_yzx: tuple[int, int, int] = (660, 512, 512)`) -> `NDArray`

Pure function boundary: Ingests memory-loaded bytes and metadata parameters. Never calls Python ``open()`` or OS file system access.

Contracts:

- require: `len(byte_stream) == shape_yzx * shape_yzx * shape_yzx * 2`
- ensure: `result.dtype == np.float32` (due to scale factor multiplication)
- ensure: `result.shape == (shape_yzx, shape_yzx, shape_yzx)`

Witness: Pass a zeroed byte stream of appropriate length. Verify output shape is correctly transposed to (512, 660, 512) and array consists of zeros.

Dependencies: `numpy`

CDG stages covered: `passenger_screening/proprietary_conversion`

Conclusion

The architecture of robust, reproducible computational directed acyclic graphs heavily relies on isolating systemic side effects from the core mathematical transformations. By structuring complex medical imaging operations—ranging from DICOM tensor algebra and Macenko optical density factorizations, to volumetric bounding box geometry and continuous coordinate mappings—strictly as stateless, pure functions mapped over highly optimized numpy and scipy.ndimage backends, data scientists and computational biologists can drastically improve testability. The defined atoms establish a comprehensive reference library for medical image processing, enforcing rigorous interface contracts while providing best-in-class algorithmic implementations capable of handling the severe constraints of memory scale, architectural anisotropy, and physical hardware noise inherent in the high-stakes medical domain.

Works cited

1. `Python-DICOM-Processing-And-Segmentation/3D-Visualisation.ipynb` at master - GitHub, accessed April 29, 2026, <https://github.com/shliamin/Python-DICOM-Processing-And-Segmentation/blob/master/3D-Visualisation.ipynb>

2. Downloading and Preprocessing Medical Images in Bulk: DICOM to NumPy with Python, accessed April 29, 2026,
<https://glassboxmedicine.com/2021/02/16/downloading-and-preprocessing-medical-images-in-bulk-dicom-to-numpy-with-python/>
3. Dealing with DICOM Using ImageIO Python Package - Towards Data Science, accessed April 29, 2026,
<https://towardsdatascience.com/dealing-with-dicom-using-imageio-python-package-117f1212ab82/>
4. VOI LUT Function Attribute – DICOM Standard Browser, accessed April 29, 2026,
<https://dicom.innolitics.com/ciods/parametric-map/parametric-map-multi-frame-functional-groups/52009229/00289132/00281056>
5. Clarify exact windowing function - DICOM, accessed April 29, 2026,
https://dicom.nema.org/Dicom/News/jan2014/docs_jan2014/cp1316.pdf
6. C.11.2 VOI LUT Module - DICOM, accessed April 29, 2026,
https://dicom.nema.org/medical/dicom/2022e/output/chtml/part03/sect_C.11.2.html
7. How to create a list of DICOM files and convert it to a single numpy array .npy?, accessed April 29, 2026,
<https://stackoverflow.com/questions/75035697/how-to-create-a-list-of-dicom-files-and-convert-it-to-a-single-numpy-array-npy>
8. General resampling between images with scipy.ndimage.map_coordinates, accessed April 29, 2026,
https://bic-berkeley.github.io/psych-214-fall-2016/map_coordinates.html
9. Making SciPy's Image Interpolation Consistent and Well Documented - Quansight Labs, accessed April 29, 2026,
<https://labs.quansight.org/blog/scipy-ndimage-interpolation>
10. FONDUE: Robust resolution-invariant denoising of MR images using Nested UNets, accessed April 29, 2026,
https://direct.mit.edu/imag/article/doi/10.1162/imag_a_00374/125037/FONDUE-Robust-resolution-invariant-denoising-of-MR
11. Multidimensional Image Processing (scipy.ndimage) — SciPy v1.17.0 Manual, accessed April 29, 2026, <https://docs.scipy.org/doc/scipy/tutorial/ndimage.html>
12. scipy.ndimage.interpolation.zoom uses nearest-neighbor-like algorithm for scaling-down, accessed April 29, 2026,
<https://stackoverflow.com/questions/34047874/scipy-ndimage-interpolation-zoom-uses-nearest-neighbor-like-algorithm-for-scalin>
13. cuCIM API Reference - RAPIDS Docs, accessed April 29, 2026,
<https://docs.rapids.ai/api/cucim/stable/api/>
14. skimage.transform — skimage 0.26.0 documentation, accessed April 29, 2026,
<https://scikit-image.org/docs/stable/api/skimage.transform.html>
15. Introduction to 3D medical imaging for machine learning: preprocessing and augmentations, accessed April 29, 2026,
https://colab.research.google.com/drive/1fyU_YaZUO3B5qVzBJwGoYZ6XVvVLog30?usp=sharing
16. Applying deformations exercise — Functional MRI methods, accessed April 29, 2026,

- https://bic-berkeley.github.io/psych-214-fall-2016/applying_deformations_solution.html
17. scipy.ndimage.zoom bad interpolation when downsampling (zoom < 1) #7324 - GitHub, accessed April 29, 2026, <https://github.com/scipy/scipy/issues/7324>
 18. Image Operators : Image Processing in Python - hlevkin, accessed April 29, 2026, https://www.hlevkin.com/hlevkin/45pythonImageProc/Image_Operators_kinser.pdf
 19. Image Processing Masterclass with Python: 50+ Solutions and Techniques Solving Complex Digital Image Processing Challenges Using Numpy, Scipy, Pytorch and Keras (English Edition) 9789389898644, 9389898641 - DOKUMEN.PUB, accessed April 29, 2026, <https://dokumen.pub/image-processing-masterclass-with-python-50-solutions-and-techniques-solving-complex-digital-image-processing-challenges-using-numpy-sci-py-pytorch-and-keras-english-edition-9789389898644-9389898641.html>
 20. A Method for Normalizing Histology Slides for Quantitative Analysis, accessed April 29, 2026, <https://www.cs.unc.edu/~mn/sites/default/files/macenko2009.pdf>
 21. A Method for Normalizing Histology Slides for Quantitative Analysis. - ResearchGate, accessed April 29, 2026, https://www.researchgate.net/publication/221624097_A_Method_for_Normalizing_Histology_Slides_for_Quantitative_Analysis
 22. A method for normalizing pathology images to improve feature extraction for quantitative pathology - Stanford University, accessed April 29, 2026, <https://web.stanford.edu/group/rubinlab/pubs/Tam-2016-MedPhys.pdf>
 23. Stain Normalization: StainGAN, StainNet - Kaggle, accessed April 29, 2026, <https://www.kaggle.com/code/nghihuynh/stain-normalization-staingan-stainnet>
 24. Macenko Method for Normalizing Histology Slides for Quantitative Analysis - GeeksforGeeks, accessed April 29, 2026, <https://www.geeksforgeeks.org/machine-learning/macenko-method-for-normalizing-histology-slides-for-quantitative-analysis/>
 25. A High-Performance System for Robust Stain Normalization of Whole-Slide Images in Histopathology - PMC, accessed April 29, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC6778842/>
 26. FONDUE: Robust resolution-invariant denoising of MR images using Nested UNets - PMC, accessed April 29, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12315758/>
 27. Generate Rotational Maximum Intensity Projections - Algorithms - ITK Discourse, accessed April 29, 2026, <https://discourse.itk.org/t/generate-rotational-maximum-intensity-projections/3226>
 28. python - Maximum intensity projection from image stack, accessed April 29, 2026, <https://stackoverflow.com/questions/48178916/maximum-intensity-projection-from-image-stack>
 29. Fast 3D Image Rotation for 2D Views with NumPy and SciPy | by Fuher Analytics - Medium, accessed April 29, 2026, <https://medium.com/@fuherarich/fast-3d-image-rotation-for-2d-views-with-numpy-and-sci-py-3f37844d0530>

30. FONDUE: Robust resolution-invariant denoising of MR Images using Nested UNets, accessed April 29, 2026,
<https://www.biorxiv.org/content/10.1101/2023.06.04.543602v1.full-text>
31. Automated Detection and Segmentation of Ascending Aorta Dilation on a Non-ECG-Gated Chest CT Using Deep Learning - PMC, accessed April 29, 2026,
<https://pmc.ncbi.nlm.nih.gov/articles/PMC12468869/>
32. Decision Support System for Liver Lesion Segmentation Based on Advanced Convolutional Neural Network Architectures - MDPI, accessed April 29, 2026,
<https://www.mdpi.com/2306-5354/9/9/467>
33. NetTracer3D Enables User-Friendly Analysis of Diverse Microscopic and Medical 3D Datasets | bioRxiv, accessed April 29, 2026,
<https://www.biorxiv.org/content/10.64898/2026.03.25.714104v1.full-text>
34. Slice-Fusion: Reducing False Positives in Liver Tumor Detection for Mask R-CNN, accessed April 29, 2026,
<https://www.computer.org/csdl/journal/tb/2023/05/10097457/1M9Iz5FbVWU>
35. Deep learning software and revised 2D model to segment bone in micro-CT scans - PMC, accessed April 29, 2026,
<https://pmc.ncbi.nlm.nih.gov/articles/PMC12868216/>
36. Multicut Algorithms for Neurite Segmentation - Heidelberg University, accessed April 29, 2026,
http://archiv.ub.uni-heidelberg.de/volltextserver/24596/1/thesis_thorsten_beier.pdf
37. Domain Explanation, Visualization, & Modeling - Kaggle, accessed April 29, 2026,
<https://www.kaggle.com/code/dhuang718/domain-explanation-visualization-modeling>
38. TRenDS Neuroimaging - Data Analysis & 3D Features - Kaggle, accessed April 29, 2026,
<https://www.kaggle.com/code/gunesevitan/trends-neuroimaging-data-analysis-3d-features>
39. Exploratory Data Analysis of fMRI _ NeuroImaging - Kaggle, accessed April 29, 2026,
<https://www.kaggle.com/code/deepthyrosejose/exploratory-data-analysis-of-fmri-neuroimaging>
40. Crop non-overlapping regions with a 3D mask - fmri - Neurostars, accessed April 29, 2026,
<https://neurostars.org/t/crop-non-overlapping-regions-with-a-3d-mask/22598>
41. Kaggle Winning Solutions: AI Trends & Insights, accessed April 29, 2026,
<https://www.kaggle.com/code/tahaalselwii/kaggle-winning-solutions-ai-trends-insights>
42. Mask a three dimensional array to perform segmentation - Stack Overflow, accessed April 29, 2026,
<https://stackoverflow.com/questions/62586686/mask-a-three-dimensional-array-to-perform-segmentation>
43. Crop 3D image based om 2D mask in python using numpy and opencv - Stack Overflow, accessed April 29, 2026,
<https://stackoverflow.com/questions/63044577/crop-3d-image-based-om-2d-mask->

[in-python-using-numpy-and-opencv](#)

44. Machine Learning, Intelligent Data Analysis and Artificial Intelligence of Things - Вінницький Національний Технічний Університет, accessed April 29, 2026, <https://iq.vntu.edu.ua/repository/getfile.php/9462.pdf>
45. Improving Airport Security with IoT-Powered Deep Learning Methods for Threat Detection and Intelligent Recommendation Systems - ResearchGate, accessed April 29, 2026, https://www.researchgate.net/publication/390111845_Improving_Airport_Security_with_IoT-Powered_Deep_Learning_Methods_for_Threat_Detection_and_Intelligent_Recommendation_Systems
46. SciPy - ndimage.label() Function - TutorialsPoint, accessed April 29, 2026, https://www.tutorialspoint.com/scipy/scipy_ndimage_label_function.htm
47. A Beginner's Guide to Scipy.ndimage - Pierian Training, accessed April 29, 2026, <https://pieriantraining.com/a-beginners-guide-to-scipyndimage/>
48. How to find the largest connected region using scipy.ndimage? - Stack Overflow, accessed April 29, 2026, <https://stackoverflow.com/questions/53576830/how-to-find-the-largest-connected-region-using-scipy-ndimage>
49. Passenger Screening Algorithm Challenge - Kaggle, accessed April 29, 2026, <https://www.kaggle.com/c/passenger-screening-algorithm-challenge/data>
50. [Kaggle] Passenger Screening Algorithm Challenge 銀牌分享| by Andrew Wang | 人工智慧的開發與落地| Medium, accessed April 29, 2026, <https://medium.com/%E4%BA%BA%E5%B7%A5%E6%99%BA%E6%85%A7%E7%9A%84%E9%96%8B%E7%99%BC%E8%88%87%E8%90%BD%E5%9C%B0/kaggle-passenger-screening-algorithm-challenge-%E9%8A%80%E7%89%8C%E5%88%86%E4%BA%AB-38329fbb8852>
51. Passenger Screening Algorithm Challenge - Kaggle, accessed April 29, 2026, <https://www.kaggle.com/competitions/passenger-screening-algorithm-challenge/discussion/35116>
52. 3D Cad to Binary Voxel Numpy Array | by Vishwajeet Sawant | Analytics Vidhya - Medium, accessed April 29, 2026, <https://medium.com/analytics-vidhya/3d-cad-to-binary-voxel-numpy-array-b538d00d97da>